WEATHER DATA PIPELINE PROJECT REPORT

1. Project Title

Complete Weather Data Pipeline System using Python, SQLite & OpenWeather API

2. Objective

The objective of this project is to build an end-to-end data engineering pipeline that:

- Extracts real-time weather data from OpenWeather API
- Transforms and validates the data
- Loads it into SQLite database
- Generates reports and monitoring alerts
- Provides dashboard visualization

3. Tools & Technologies Used

- Python
- SQLite Database
- Pandas
- Logging Module
- Streamlit (Dashboard)
- Schedule Library

4. System Architecture

API (OpenWeather)

↓

API Client (api_client.py)

↓

Validation Layer (validators.py)

↓

ETL Pipeline (etl_pipeline.py)

↓

SQLite Database (weather.db)

↓

Reporting Module (reporter.py)

↓

Monitoring System (monitor.py)

↓

Streamlit Dashboard (dashboard.py)

5. Database Design

Tables used:

1. cities

- city_id (PK)
- city_name
- country

2. weather

- weather_id (PK)
- city_id (FK)
- temperature
- humidity
- condition
- timestamp

3. api_logs

- log_id (PK)
- city
- status
- error_message
- timestamp

6. ETL Process

Extract:

- Data fetched using OpenWeather API

Transform:

- Validate temperature range
- Validate humidity range
- Extract required fields

Load:

- Store into SQLite database

7. Features Implemented

✓ Real-time API integration
✓ Data validation system
✓ Logging system (error + info logs)
✓ Automated ETL pipeline
✓ Scheduler (runs every 10 sec / hourly)
✓ Monitoring system (alerts)
✓ Reporting system (text file report)
✓ Dashboard visualization

8. Dashboard (Streamlit)

The dashboard displays:

- Live weather data per city
- Temperature trends
- Humidity levels
- System health status
- Latest updates from database

9. Results and System Outputs

This section demonstrates the working of the Weather Data Pipeline System through real execution outputs, database records, and dashboard visualization.

9.1 API Execution Output

The system successfully fetches real-time weather data from the OpenWeather API.

Output:

- City-wise weather data retrieved successfully
- Temperature, humidity, pressure, and weather conditions extracted
- API errors handled using exception handling

```
🌐 Testing API...

✅ API Working!
{'city': 'Delhi', 'temperature': 37.05, 'humidity': 19, 'pressure': 1005, 'wind_speed': 5.14, 'condition':
 'haze', 'timestamp': datetime.datetime(2026, 4, 22, 11, 32, 39, 210421)}
```

9.2 ETL Pipeline Execution

The ETL pipeline extracts data from API, transforms it using validation rules, and loads it into SQLite database.

Observations:

- Data processed for multiple cities
- Invalid records filtered using validation checks
- Logs generated for each ETL cycle

```
2026-04-22 11:50:39,249 - INFO - 🚀 ETL Pipeline Started
2026-04-22 11:50:39,249 - INFO - 📡 Processing: Delhi
2026-04-22 11:50:40,133 - INFO - ✅ Inserted data for Delhi
2026-04-22 11:50:40,133 - INFO - 📡 Processing: Mumbai
2026-04-22 11:50:40,874 - INFO - ✅ Inserted data for Mumbai
2026-04-22 11:50:40,875 - INFO - 📡 Processing: Chennai
2026-04-22 11:50:41,642 - INFO - ✅ Inserted data for Chennai
2026-04-22 11:50:41,642 - INFO - 📡 Processing: Bangalore
2026-04-22 11:50:42,417 - INFO - ✅ Inserted data for Bangalore
2026-04-22 11:50:42,417 - INFO - 🏁 ETL Pipeline Completed
```

9.3 Database Records (SQLite)

The processed weather data is stored in a normalized SQLite database with 3 tables:

- cities
- weather

Observations:

- Data stored successfully after ETL execution
- Foreign key relationships maintained
- Historical data available for analysis

Rows: 4

	city_id	city_name	country
1	1	Bengaluru	IN
2	2	Delhi	IN
3	3	Mumbai	IN
4	4	Chennai	IN
5			

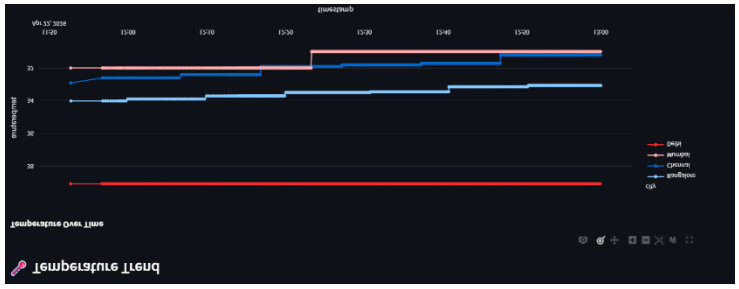
	weath...	city_id	temperat...	humidity	weather_...	timestamp
1	1	1	28.5	46	few clouds	2026-04-21 17:...
2	2	2	32.05	29	haze	2026-04-21 17:...
3	3	3	30.99	58	smoke	2026-04-21 17:...
4	4	4	29.97	83	scattered clouds	2026-04-21 17:...
5	5	1	28.5	46	few clouds	2026-04-21 17:...
6	6	2	32.05	29	haze	2026-04-21 17:...
7	7	3	30.99	58	smoke	2026-04-21 17:...
8	8	4	29.97	83	scattered clouds	2026-04-21 17:...
109	9	1	28.5	46	few clouds	2026-04-21 17:...

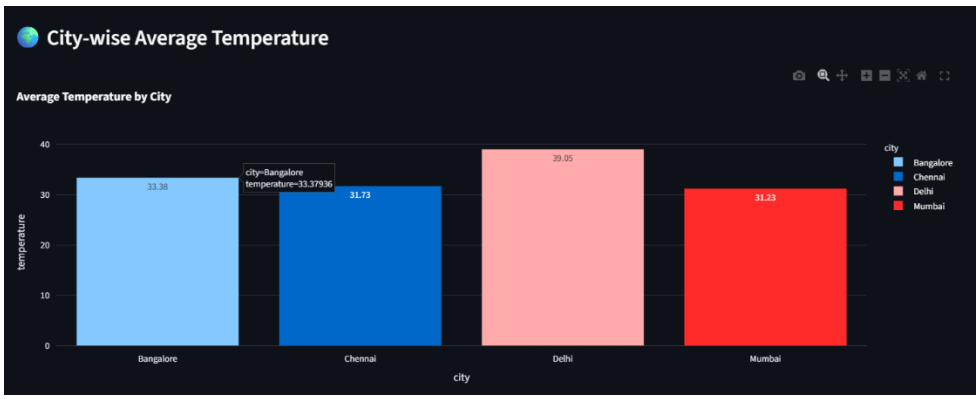
9.4 Dashboard Visualization

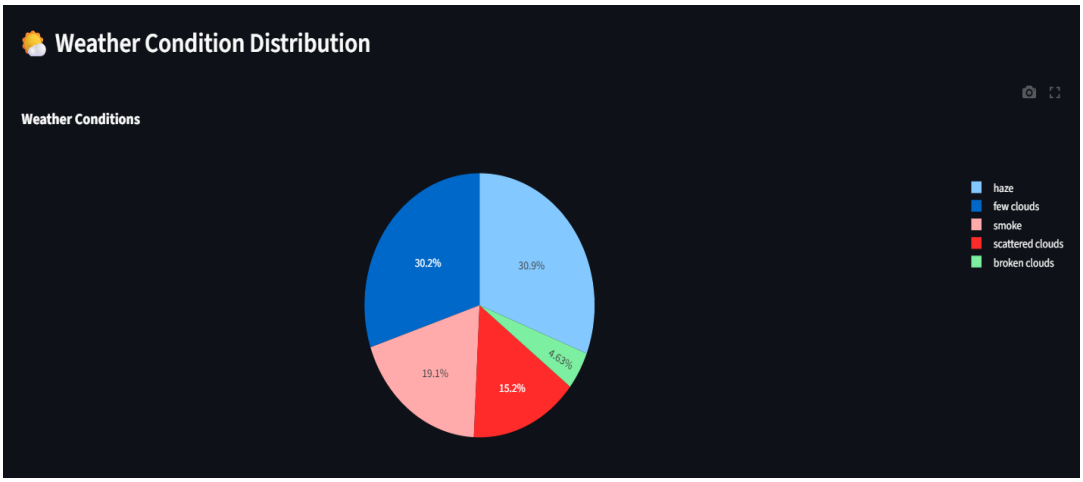
A Streamlit-based dashboard is developed to visualize live weather data.

Features:

- Real-time weather display
- City-wise temperature and humidity
- Latest updates from database







9.5 Report Generation

Automated reports are generated using Python and stored in the /reports folder.

Observations:

- Latest weather snapshot included
- Maximum and average temperature calculated
- Report saved as text file

reports > weather_report.txt

```
🌥️ WEATHER REPORT
Generated at: 2026-04-21 21:00:44
========================================

📍 Latest Weather (Per City):
Delhi: 33.05°C | 25% | haze | 2026-04-21 15:30:41
Mumbai: 30.99°C | 58% | smoke | 2026-04-21 15:30:42
Chennai: 29.99°C | 82% | few clouds | 2026-04-21 15:30:43
Bengaluru: 29.72°C | 33% | few clouds | 2026-04-21 15:30:43

📊 Summary:
🔥 Max Temperature: 39.05°C
🌡️ Average Temperature: 34.39°C

✅ End of Report
```

9.6 Logging System

A logging system is implemented to track application activity.

Logs include:

- API success/failure logs
- ETL pipeline status logs
- Error tracking logs

```
2026-04-21 22:42:33,331 - INFO - Application started
2026-04-21 22:42:33,331 - INFO - Scheduler initialized
2026-04-22 17:26:42,634 - INFO - 🚀 Weather Pipeline System Starting...
2026-04-22 17:26:48,029 - INFO - ✅ System initialized successfully
2026-04-22 17:28:52,206 - INFO - 🚀 Weather Pipeline System Starting...
2026-04-22 17:28:59,864 - INFO - ✅ System initialized successfully
2026-04-22 17:37:16,108 - INFO - 🚀 Weather Pipeline System Starting...
2026-04-22 17:37:20,869 - INFO - ✅ System initialized successfully
2026-04-22 17:39:56,277 - INFO - System health check completed
2026-04-22 17:41:01,436 - INFO - Report generated successfully
2026-04-22 17:42:02,001 - INFO - Application started
2026-04-22 17:42:02,001 - INFO - Scheduler initialized
2026-04-22 17:43:48,849 - INFO - Application started
2026-04-22 17:43:48,849 - INFO - Scheduler initialized
2026-04-22 17:49:26,567 - INFO - 🚀 Weather Pipeline System Starting...
2026-04-22 17:49:26,570 - INFO - System health check completed
2026-04-22 17:49:26,577 - INFO - Report generated successfully
2026-04-22 17:49:29,609 - INFO - ✅ System initialized successfully
2026-04-22 17:51:14,613 - INFO - System health check completed
2026-04-22 17:51:24,729 - INFO - Report generated successfully
2026-04-22 17:53:58,578 - INFO - Application started
2026-04-22 17:53:58,579 - INFO - Scheduler initialized
2026-04-22 17:56:32,352 - INFO - Report generated successfully
2026-04-22 18:00:31,148 - INFO - TEST: App logger working
```

settings.json | init_db.py | logger.py | dashboard.py 3 | weather_report.txt | error.log

logs > error.log

```
2026-04-22 16:14:11,494 - ERROR - ❌ ETL Failed: fetch_weather() missing 1 required positional argum
2026-04-22 16:15:53,788 - ERROR - ❌ ETL Failed: fetch_weather() missing 1 required positional argum
2026-04-22 16:24:55,795 - ERROR - ❌ ETL Failed: table weather has no column named city
2026-04-22 16:35:31,800 - ERROR - ❌ ETL Failed: table weather has no column named city
2026-04-22 16:36:25,314 - ERROR - ❌ ETL Failed: table weather has no column named city
2026-04-22 16:37:12,077 - ERROR - ❌ ETL Failed: table weather has no column named city
2026-04-22 16:47:52,699 - ERROR - ❌ DB Error for city_id 1: table weather has no column named city
2026-04-22 16:47:53,406 - ERROR - ❌ DB Error for city_id 2: table weather has no column named city
2026-04-22 16:47:54,120 - ERROR - ❌ DB Error for city_id 3: table weather has no column named city
2026-04-22 16:47:54,841 - ERROR - ❌ DB Error for city_id 4: table weather has no column named city
2026-04-22 16:48:24,977 - ERROR - ❌ DB Error for city_id 1: table weather has no column named city
2026-04-22 16:48:25,637 - ERROR - ❌ DB Error for city_id 2: table weather has no column named city
2026-04-22 16:48:26,294 - ERROR - ❌ DB Error for city_id 3: table weather has no column named city
2026-04-22 16:48:26,975 - ERROR - ❌ DB Error for city_id 4: table weather has no column named city
2026-04-22 17:11:40,756 - ERROR - ❌ ETL Failed: table api_logs has no column named city_id
2026-04-22 17:15:26,546 - ERROR - ❌ DB Error for Delhi: table weather has no column named city
2026-04-22 17:15:27,417 - ERROR - ❌ DB Error for Mumbai: table weather has no column named city
2026-04-22 17:15:28,301 - ERROR - ❌ DB Error for Chennai: table weather has no column named city
2026-04-22 17:15:29,323 - ERROR - ❌ DB Error for Bangalore: table weather has no column named city
2026-04-22 18:00:31,149 - ERROR - TEST: Error logger working
```

```
2026-04-21 22:42:43,337 - INFO - 🚀 ETL Pipeline Started
2026-04-21 22:42:43,337 - INFO - 🔄 Processing: Delhi
2026-04-21 22:42:54,489 - WARNING - ❌ Invalid data for Delhi
2026-04-21 22:42:54,491 - INFO - 🔄 Processing: Mumbai
2026-04-21 22:43:17,611 - WARNING - ❌ Invalid data for Mumbai
2026-04-21 22:43:17,613 - INFO - 🔄 Processing: Chennai
2026-04-21 22:43:37,662 - WARNING - ❌ Invalid data for Chennai
2026-04-21 22:43:37,662 - INFO - 🔄 Processing: Bangalore
2026-04-21 22:43:42,903 - INFO - ✅ Inserted data for Bangalore
2026-04-21 22:43:42,903 - INFO - 🏁 ETL Pipeline Completed
2026-04-21 22:43:52,941 - INFO - 🚀 ETL Pipeline Started
2026-04-21 22:43:52,941 - INFO - 🔄 Processing: Delhi
2026-04-21 22:43:55,045 - INFO - ✅ Inserted data for Delhi
2026-04-21 22:43:55,052 - INFO - 🔄 Processing: Mumbai
2026-04-21 22:43:56,521 - INFO - ✅ Inserted data for Mumbai
2026-04-21 22:43:56,521 - INFO - 🔄 Processing: Chennai
2026-04-21 22:43:58,211 - INFO - ✅ Inserted data for Chennai
2026-04-21 22:43:58,211 - INFO - 🔄 Processing: Bangalore
2026-04-21 22:43:59,897 - INFO - ✅ Inserted data for Bangalore
2026-04-21 22:43:59,919 - INFO - 🏁 ETL Pipeline Completed
2026-04-21 22:44:09,932 - INFO - 🚀 ETL Pipeline Started
2026-04-21 22:44:09,932 - INFO - 🔄 Processing: Delhi
2026-04-21 22:44:11,410 - INFO - ✅ Inserted data for Delhi
2026-04-21 22:44:11,410 - INFO - 🔄 Processing: Mumbai
2026-04-21 22:44:12,909 - INFO - ✅ Inserted data for Mumbai
2026-04-21 22:44:12,909 - INFO - 🔄 Processing: Chennai
2026-04-21 22:44:14,832 - INFO - ✅ Inserted data for Chennai
2026-04-21 22:44:14,832 - INFO - 🔄 Processing: Bangalore
2026-04-21 22:44:16,468 - INFO - ✅ Inserted data for Bangalore
```

10. Challenges Faced

- API errors (401 Unauthorized)
- Logging Unicode errors
- Database table missing issues
- Module import path errors

11. Solutions Implemented

- Fixed API key configuration
- Created centralized logging system
- Proper database initialization script
- Structured project into modules

12. Conclusion

This project successfully demonstrates a complete data engineering pipeline with real-time API integration, ETL workflow, database storage, and dashboard visualization.